to the empirical distribution $Q(x)$. We will weight each log-likelihood ratio by its $Q(x)$ probability.

Table 7.6. The weighted log-likelihood ratio in math and Python

Math	Python

$LR = \sum_i Q(x_i) \cdot \log\left(\frac{P(x_i)}{Q(x_i)}\right)$	`p = np.array([0.1,0.1])`
	`q = np.array([0.6,0.5])`
	`def lr(p,q):`
	`  x = q * np.log(p/q)`
	`  x = np.sum(x)`
	`  return x`

We now have an objective function that measures how likely a sample from the generative model is, compared to the real-world distribution of data, weighted by how likely the sample is in the real world.

There's one last minor problem. This objective function must be maximized because we want the log-likelihood ratio to be high, but by convenience and convention we prefer to have objective functions that are error or loss functions to be minimized. We can remedy this by adding a negative sign, so a high likelihood ratio becomes a small error or loss.

Table 7.7. The Kullback-Leibler divergence

Math	Python

$D_{KL}(Q \parallel P) = -\sum_i Q(x_i) \cdot \log\left(\frac{P(x_i)}{Q(x_i)}\right)$	`p = np.array([0.1,0.1])`
	`q = np.array([0.6,0.5])`
	`def lr(p,q):`
	`  x = q * np.log(p/q)`
	`  x = -1 * np.sum(x)`
	`  return x`

You may notice we switched out *LR* for some strange symbols: $D_{KL}(Q \parallel P)$. It turns out the objective function we just created is a very important one in all of machine learning; it's called the *Kullback-Leibler divergence* (KL divergence for short). The KL divergence is a kind of error function